

JavaScript Basics for Playwright & Test Frameworks

A practical guide to the **JavaScript you need** to write Playwright tests and build a maintainable automation framework (Page Object Model, fixtures, helpers, data-driven tests).

Who this is for: Testers and developers who know some HTML/CSS and want solid JS for `@playwright/test`.

Part 1 — Why JavaScript Matters for Playwright

1.1 What Playwright tests are written in

Playwright Test uses **JavaScript** (or TypeScript). Every test file is `.js` / `.ts` and runs on **Node.js**.

```
const { test, expect } = require('@playwright/test');

test('login page loads', async ({ page }) => {
  await page.goto('/login');
  await expect(page.getByRole('heading', { name: 'Sign in' })).toBeVisible();
});
```

JS concept	Where you use it in Playwright
<code>async</code> / <code>await</code>	Almost every action (<code>goto</code> , <code>click</code> , <code>fill</code>)
Objects	Page Objects, config, test data
Functions / classes	Helpers, POM methods
Arrays	Test data, <code>test.describe</code> loops
Modules (<code>require</code> / <code>import</code>)	Split pages, utils, fixtures
Promises / errors	Network waits, try/catch around APIs

1.2 Mental model: browser automation is async

Clicking a button is **not instant**. Playwright methods return **Promises**. You must `await` them or the next line runs too early.

```
// WRONG – race, flaky
page.click('#login');
page.fill('#otp', '123456');

// RIGHT
await page.click('#login');
await page.fill('#otp', '123456');
```

Part 2 – Variables, Types, and Operators

2.1 Prefer `const` and `let`

Keyword	Use when
<code>const</code>	Binding never reassigned (default choice)
<code>let</code>	Value changes later (counters, flags)
<code>var</code>	Avoid in new Playwright code

```
const BASE_URL = 'https://uat.example-bank.com';
let attempt = 0;
attempt += 1;
```

`const` objects can still have **properties changed**:

```
const user = { name: 'Ada' };
user.name = 'Grace'; // OK
// user = {};          // Error – cannot reassign const
```

2.2 Types you see in tests

```
typeof 42;           // "number"
typeof 'hi';         // "string"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof null;         // "object" (quirk)
typeof {};           // "object"
typeof [];           // "object"
Array.isArray([]);   // true – use this for arrays
```

2.3 Equality – always `===`

```
'5' == 5;   // true  (coercion – avoid)
'5' === 5;  // false (strict – prefer)
```

2.4 Truthy / falsy (useful for flags)

Falsy: `false`, `0`, `""`, `null`, `undefined`, `NaN`

Truthy: everything else (`"0"`, `[]`, `{}`)

```
const email = process.env.TEST_EMAIL || 'demo@bank.com';
const timeout = process.env.TIMEOUT ?? 30000; // ?? only replaces null/undefined
```

2.5 Template literals (URLs, messages)

```
const accountId = 'ACC-1001';
await page.goto(`${BASE_URL}/accounts/${accountId}`);
await expect(page.getByText(`Account ${accountId}`)).toBeVisible();
```

Part 3 – Functions (Core of Every Framework)

3.1 Function declaration and expression

```
function sum(a, b) {
  return a + b;
}

const multiply = function (a, b) {
  return a * b;
};
```

3.2 Arrow functions (everywhere in Playwright)

```
const double = (n) => n * 2;

const users = [{ name: 'A' }, { name: 'B' }];
const names = users.map((u) => u.name);
```

Arrow functions are ideal as short callbacks. They do **not** get their own `this` (relevant for methods on objects – use regular methods in Page Objects).

3.3 Default and rest parameters

```
function openAccount(page, product = 'savings') {
  return page.goto(`/open/${product}`);
}

function joinTags(...tags) {
  return tags.join(',');
}
```

3.4 Async functions (required pattern)

Any Playwright action that talks to the browser is async. Framework helpers should usually be `async` too.

```

async function login(page, username, password) {
  await page.getByLabel('Username').fill(username);
  await page.getByLabel('Password').fill(password);
  await page.getByRole('button', { name: 'Sign in' }).click();
}

test('customer logs in', async ({ page }) => {
  await page.goto('/login');
  await login(page, 'demo', 'secret');
  await expect(page).toHaveURL(/dashboard/);
});

```

If you forget `await` on an async helper, the test may finish before login completes.

Part 4 — Promises, async/await, and Errors

4.1 What a Promise is

A Promise is a value that finishes later: **pending** → **fulfilled** or **rejected**.

```

const p = page.title(); // Promise<string>
const title = await p;  // wait until it settles

```

4.2 `async` / `await` rules

1. Only use `await` inside an `async` function (or at top level in modules).
2. `await` waits for the Promise; you get the resolved value.
3. Errors rejected by the Promise become **throwable** — catch with `try/catch`.

```

async function safeGoto(page, url) {
  try {
    await page.goto(url, { waitUntil: 'domcontentloaded' });
    return true;
  } catch (err) {
    console.error('Navigation failed:', err.message);
    return false;
  }
}

```

4.3 Parallel awaits (when independent)

```

// Sequential (slower)
const t1 = await page.locator('#a').textContent();
const t2 = await page.locator('#b').textContent();

// Parallel (faster) when independent
const [t1, t2] = await Promise.all([
  page.locator('#a').textContent(),
  page.locator('#b').textContent(),
]);

```

Do **not** parallelize dependent steps (fill password before username finishes).

4.4 Common async mistakes

Mistake	Fix
Missing <code>await</code> on <code>click</code> / <code>fill</code>	Always await Playwright actions
<code>async</code> helper without <code>await</code> at call site	<code>await login(page, ...)</code>
Using <code>.then</code> chains everywhere	Prefer <code>async</code> / <code>await</code> for readability

Part 5 – Objects and Destructuring

5.1 Objects as config and test data

```
const credentials = {
  valid: { user: 'cust01', pass: 'Pass@123' },
  locked: { user: 'locked', pass: 'x' },
};

await page.fill('#user', credentials.valid.user);
```

5.2 Destructuring (fixtures and returns)

Playwright fixtures use destructuring heavily:

```
test('example', async ({ page, context, browserName }) => {
  // page, context come from the test runner
  console.log(browserName);
});
```

Your own returns:

```
function readBalanceCard(el) {
  return { currency: 'INR', amount: 1000 };
}

const { currency, amount } = readBalanceCard();
```

5.3 Spread and merge (config)

```
const defaults = { headless: true, timeout: 30000 };
const local = { headless: false };
const config = { ...defaults, ...local }; // headless: false
```

5.4 Optional chaining and nullish coalescing

```
const city = response.body?.customer?.address?.city;  
const retries = options.retries ?? 2;
```

Stops crashes when nested data is missing – useful parsing API JSON in hybrid UI+API tests.

Part 6 – Arrays (Data-Driven Tests)

6.1 Create and access

```
const products = ['savings', 'current', 'fd'];  
console.log(products[0]);  
console.log(products.length);  
console.log(products.at(-1)); // last
```

6.2 Methods used daily in frameworks

```
const amounts = [100, 250, 80];  
  
const doubled = amounts.map((n) => n * 2);  
const big = amounts.filter((n) => n >= 100);  
const total = amounts.reduce((sum, n) => sum + n, 0);  
  
const found = users.find((u) => u.id === 2);  
const ok = statuses.every((s) => s === 'SUCCESS');
```

6.3 Data-driven test loops

```
const cases = [  
  { user: 'ok', pass: 'ok', expect: '/dashboard/' },  
  { user: 'bad', pass: 'x', expect: '/error/' },  
];  
  
for (const c of cases) {  
  test(`login: ${c.user}`, async ({ page }) => {  
    await page.goto('/login');  
    await page.fill('#user', c.user);  
    await page.fill('#pass', c.pass);  
    await page.click('button[type=submit]');  
    await expect(page).toHaveURL(c.expect);  
  });  
}
```

6.4 for...of vs forEach

Prefer `for...of` when you need `await` inside the loop:

```
// forEach does NOT wait for async correctly – avoid for Playwright steps
for (const item of items) {
  await page.getByText(item).click();
}
```

Part 7 – Classes and Page Object Model (POM)

7.1 Why classes for frameworks

A **Page Object** wraps locators and actions for one screen so tests stay short and reuse is easy.

7.2 Class basics

```
class LoginPage {
  constructor(page) {
    this.page = page;
    this.userInput = page.getByLabel('Username');
    this.passInput = page.getByLabel('Password');
    this.submitBtn = page.getByRole('button', { name: 'Sign in' });
  }

  async open() {
    await this.page.goto('/login');
  }

  async login(username, password) {
    await this.userInput.fill(username);
    await this.passInput.fill(password);
    await this.submitBtn.click();
  }
}

module.exports = { LoginPage };
```

7.3 Using the Page Object in a test

```
const { test, expect } = require('@playwright/test');
const { LoginPage } = require('../pages/LoginPage');

test('customer can login', async ({ page }) => {
  const login = new LoginPage(page);
  await login.open();
  await login.login('demo', 'secret');
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

7.4 Method return `this` for chaining (optional)

```
async fillUser(u) {
  await this.userInput.fill(u);
  return this;
}
// await login.fillUser('a').fillPass('b'); // if chainable
```

Keep chaining light — clarity beats clever chains.

7.5 What belongs where

Layer	Contains
Test (<code>*.spec.js</code>)	Arrange / act / assert story only
Page Object	Locators + user actions for one page
Helper / util	Shared pure JS (format date, read env)
Fixture	Inject custom objects into <code>test(...)</code>
Config	Browsers, baseUrl, retries, reporters

Part 8 — Modules (`require` / `import`)

8.1 CommonJS (default in many Playwright JS projects)

```
// utils/env.js
function getBaseUrl() {
  return process.env.BASE_URL || 'http://localhost:3000';
}
module.exports = { getBaseUrl };

// test file
const { getBaseUrl } = require('../utils/env');
```

8.2 ES Modules (if `"type": "module"` or `.mjs`)

```
export function getBaseUrl() {
  return process.env.BASE_URL || 'http://localhost:3000';
}

import { getBaseUrl } from '../utils/env.js';
```

Stick to **one style** per project. Playwright's generated JS projects often use CommonJS; TypeScript projects use ES imports.

8.3 Folder structure (framework skeleton)

```
project/  
  playwright.config.js  
  package.json  
  tests/  
    login.spec.js  
    transfer.spec.js  
  pages/  
    LoginPage.js  
    DashboardPage.js  
  utils/  
    env.js  
    testdata.js  
  fixtures/  
    bank.fixture.js
```

Part 9 — Playwright-Specific JavaScript Patterns

9.1 Fixtures and destructuring

Built-in fixtures: `page` , `context` , `browser` , `request` , `baseURL` .

```
test('api health', async ({ request }) => {  
  const res = await request.get('/api/health');  
  expect(res.ok()).toBeTruthy();  
});
```

Custom fixture example (concept):

```
const { test as base } = require('@playwright/test');  
const { LoginPage } = require('../pages/LoginPage');  
  
const test = base.extend({  
  loginPage: async ({ page }, use) => {  
    await use(new LoginPage(page));  
  },  
});  
  
test('with fixture', async ({ loginPage }) => {  
  await loginPage.open();  
});
```

9.2 Locators are sync; actions are async

```
const btn = page.getByRole('button', { name: 'Pay' }); // locator object (sync)  
await btn.click(); // action (async) – must await
```

9.3 Assertions with `expect`

```
await expect(page).toHaveURL(/accounts/);
await expect(page.getByText('Balance')).toBeVisible();
expect(array.length).toBe(3); // sync expect for pure JS values
```

UI assertions from Playwright are async (`await expect(...)`). Pure value checks can be sync.

9.4 Soft vs hard failures (concept)

```
await expect.soft(page.getByText('Fee')).toBeVisible();
await expect.soft(page.getByText('Tax')).toBeVisible();
// both checked; test fails at end if soft failed
```

9.5 Environment variables

```
// .env or CI variables
const user = process.env.BANK_USER;
const pass = process.env.BANK_PASS;
```

Never hardcode production secrets in Git. Use CI secrets or local env files (gitignored).

Part 10 — Conditionals, Loops, and Test Control

10.1 Skip / fixme based on JS conditions

```
test.skip(!process.env.RUN_SLOW, 'slow suite off');
test('transfer', async ({ page }) => { /* ... */ });
```

10.2 `test.describe` for grouping

```
test.describe('Net banking login', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
  });

  test('valid user', async ({ page }) => { /* ... */ });
  test('locked user', async ({ page }) => { /* ... */ });
});
```

10.3 Hooks are async too

```
test.beforeAll(async () => {
  // seed data via API once
});

test.afterEach(async ({ page }, testInfo) => {
  if (testInfo.status !== testInfo.expectedStatus) {
    await page.screenshot({ path: `fail-${testInfo.title}.png` });
  }
});
```

Part 11 — Strings, Numbers, Dates, JSON

11.1 Strings for assertions and parsing

```
const text = await page.locator('#balance').innerText();
const cleaned = text.replace(/^[^0-9.]/g, '');
const amount = Number(cleaned);
expect(amount).toBeGreaterThan(0);
```

11.2 JSON for API + UI hybrids

```
const body = await response.json();
expect(body.status).toBe('SUCCESS');
expect(body.txnId).toMatch(/^TXN/);
```

```
const payload = { amount: 100, currency: 'INR' };
const json = JSON.stringify(payload);
const copy = JSON.parse(json);
```

11.3 Dates

```
const today = new Date().toISOString().slice(0, 10); // YYYY-MM-DD
await page.fill('#valueDate', today);
```

Part 12 — Error Handling Best Practices

12.1 Fail fast in tests

Prefer Playwright auto-waiting + `expect` over custom retries that hide bugs.

12.2 Catch when calling flaky external systems

```
async function createCustomer(request) {
  const res = await request.post('/api/customers', { data: { name: 'T' } });
  if (!res.ok()) {
    throw new Error(`Create failed: ${res.status()} ${await res.text()}`);
  }
  return res.json();
}
```

Throwing clear errors makes CI logs usable.

Part 13 — End-to-End Framework Mini Example

13.1 pages/TransferPage.js

```
class TransferPage {
  constructor(page) {
    this.page = page;
    this.from = page.getByLabel('From account');
    this.to = page.getByLabel('To account');
    this.amount = page.getByLabel('Amount');
    this.submit = page.getByRole('button', { name: 'Transfer' });
  }

  async goto() {
    await this.page.goto('/transfer');
  }

  async submitTransfer({ from, to, amount }) {
    await this.from.selectOption(from);
    await this.to.fill(to);
    await this.amount.fill(String(amount));
    await this.submit.click();
  }
}

module.exports = { TransferPage };
```

13.2 utils/testdata.js

```
module.exports = {
  transferCases: [
    { from: 'ACC1', to: 'ACC2', amount: 100, ok: true },
    { from: 'ACC1', to: 'ACC2', amount: 0, ok: false },
  ],
};
```

13.3 tests/transfer.spec.js

```
const { test, expect } = require('@playwright/test');
const { TransferPage } = require('../pages/TransferPage');
const { transferCases } = require('../utils/testdata');

test.describe('Fund transfer', () => {
  for (const c of transferCases) {
    test(`amount ${c.amount}`, async ({ page }) => {
      const transfer = new TransferPage(page);
      await transfer.goto();
      await transfer.submitTransfer(c);
      if (c.ok) {
        await expect(page.getByText('Success')).toBeVisible();
      } else {
        await expect(page.getByText('Invalid amount')).toBeVisible();
      }
    });
  }
});
```

This is the standard shape of a **basic Playwright framework**: data + page objects + thin tests.

Part 14 — Quick Reference Cheat Sheet

```
// Import runner
const { test, expect } = require('@playwright/test');

// Async test + fixture
test('name', async ({ page }) => {
  await page.goto('/path');
  await page.getByRole('button', { name: 'Go' }).click();
  await expect(page).toHaveURL(/done/);
});

// Locator then action
const el = page.getByTestId('otp');
await el.fill('123456');

// Page Object
class X { constructor(page) { this.page = page; } }
const x = new X(page);
await x.someMethod();

// Data loop
for (const row of rows) {
  test(`case ${row.id}`, async ({ page }) => { /* ... */ });
}

// Env
process.env.BASE_URL

// JSON API
const data = await (await request.get('/api')).json();
```

Part 15 — Interview-Style Quick Questions

Q1. Why is almost every Playwright call `await` ed?

Because browser automation is asynchronous; methods return Promises.

Q2. Difference between locator and action?

Creating a locator is sync; clicking/filling/waiting is async.

Q3. Why use Page Object Model?

Centralize selectors and flows; keep tests readable; one place to update when UI changes.

Q4. Why avoid `forEach` with `await` ?

`forEach` does not await async callbacks; use `for...of` instead.

Q5. `const` vs `let` in frameworks?

Prefer `const` ; use `let` only when reassignment is required.

Q6. What is a fixture?

An object Playwright injects into the test function (like `page`) — you can extend with custom ones.

Q7. Sync vs async expect?

`await expect(locator)...` for UI; plain `expect(value)` for JS values.

Q8. How do you pass data into many tests?

Arrays of objects + loop creating `test(...)` , or CSV/JSON loaders.

Part 16 – Practice Checklist

- ☐ Write a helper `login(page, user, pass)` as `async` and call it with `await`
 - ☐ Build a `LoginPage` class and use it from a spec
 - ☐ Create 3 data rows and data-driven tests with `for...of`
 - ☐ Read an env var for `BASE_URL` in config or test
 - ☐ Parse `innerText` balance to a number and assert with `expect`
 - ☐ Call an API with `request` fixture and assert JSON status
 - ☐ Export/import a util with `module.exports` / `require`
-

References

1. Playwright docs – Writing tests: <https://playwright.dev/docs/writing-tests>
 2. Playwright – Page Object Model: <https://playwright.dev/docs/pom>
 3. Playwright – Fixtures: <https://playwright.dev/docs/test-fixtures>
 4. MDN – async functions: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
 5. MDN – Promises: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
 6. MDN – Classes: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes>
 7. Node.js – Modules: <https://nodejs.org/api/modules.html>
-

These notes focus on the JavaScript skills used daily in Playwright automation and framework design. Pair them with Day 8–10 Playwright notes for locators, config, mocking, and CI.